

编译原理

第一讲：绪论

授课教师：柯嵩宇

福州大学计算机与大数据学院

2026 年 3 月 5 日



福州大学
FUZHOU UNIVERSITY 1958



- ① 课程要求与成绩构成
- ② What - 编译程序是什么?
- ③ Why - 我们为什么需要编译程序?
- ④ Why to learn - 为什么要学习编译原理?
- ⑤ 编译系统概述
- ⑥ 总结与作业

目录

- ① 课程要求与成绩构成
- ② What - 编译程序是什么?
- ③ Why - 我们为什么需要编译程序?
- ④ Why to learn - 为什么要学习编译原理?
- ⑤ 编译系统概述
- ⑥ 总结与作业



课程要求、成绩构成与参考书目

- 成绩构成



课程要求、成绩构成与参考书目

- 成绩构成
 - 平时成绩：30%



课程要求、成绩构成与参考书目

- 成绩构成
 - 平时成绩：30%
 - 课前小测：15%



课程要求、成绩构成与参考书目

- 成绩构成
 - 平时成绩：30%
 - 课前小测：15%
 - 课后作业：15%



课程要求、成绩构成与参考书目

- 成绩构成
 - 平时成绩：30%
 - 课前小测：15%
 - 课后作业：15%
 - 期末考试：70%



课程要求、成绩构成与参考书目

- 成绩构成
 - 平时成绩：30%
 - 课前小测：15%
 - 课后作业：15%
 - 期末考试：70%
- 课程要求



课程要求、成绩构成与参考书目

- 成绩构成
 - 平时成绩：30%
 - 课前小测：15%
 - 课后作业：15%
 - 期末考试：70%
- 课程要求
 - 课前小测与课后作业禁用大模型



课程要求、成绩构成与参考书目

- 成绩构成
 - 平时成绩：30%
 - 课前小测：15%
 - 课后作业：15%
 - 期末考试：70%
- 课程要求
 - 课前小测与课后作业禁用大模型
 - 作业自己独立完成



课程要求、成绩构成与参考书目

● 成绩构成

- 平时成绩：30%
 - 课前小测：15%
 - 课后作业：15%
- 期末考试：70%

● 课程要求

- 课前小测与课后作业禁用大模型
- 作业自己独立完成

● 教材与参考书



课程要求、成绩构成与参考书目

● 成绩构成

- 平时成绩：30%
 - 课前小测：15%
 - 课后作业：15%
- 期末考试：70%

● 课程要求

- 课前小测与课后作业禁用大模型
- 作业自己独立完成

● 教材与参考书

- 主教材：龙书-《Compilers: Principles, Techniques, and Tools》



课程要求、成绩构成与参考书目

● 成绩构成

- 平时成绩：30%
 - 课前小测：15%
 - 课后作业：15%
- 期末考试：70%

● 课程要求

- 课前小测与课后作业禁用大模型
- 作业自己独立完成

● 教材与参考书

- 主教材：龙书-《Compilers: Principles, Techniques, and Tools》
- 参考书：

课程要求、成绩构成与参考书目

● 成绩构成

- 平时成绩：30%
 - 课前小测：15%
 - 课后作业：15%
- 期末考试：70%

● 课程要求

- 课前小测与课后作业禁用大模型
- 作业自己独立完成

● 教材与参考书

- 主教材：龙书-《Compilers: Principles, Techniques, and Tools》
- 参考书：
 - 教育部 101 计划教材-《编译原理与技术》张莉等

课程要求、成绩构成与参考书目

● 成绩构成

- 平时成绩：30%
 - 课前小测：15%
 - 课后作业：15%
- 期末考试：70%

● 课程要求

- 课前小测与课后作业禁用大模型
- 作业自己独立完成

● 教材与参考书

- 主教材：龙书-《Compilers: Principles, Techniques, and Tools》
- 参考书：
 - 教育部 101 计划教材-《编译原理与技术》张莉等
 - 虎书-《Modern Compiler Implementation in Java》

课程要求、成绩构成与参考书目

- 成绩构成
 - 平时成绩：30%
 - 课前小测：15%
 - 课后作业：15%
 - 期末考试：70%
- 课程要求
 - 课前小测与课后作业禁用大模型
 - 作业自己独立完成

- 教材与参考书
 - 主教材：龙书-《Compilers: Principles, Techniques, and Tools
 - 参考书：
 - 教育部 101 计划教材-《编译原理与技术》张莉等
 - 虎书-《Modern Compiler Implementation in Java》
 - 鲸书-《Advanced Compiler Design and Implementation》
 - 《自制编译器》，(日) 青木峰良

- 教材与参考书
 - 主教材：龙书-《Compilers: Principles, Techniques, and Tools
 - 参考书：
 - 教育部 101 计划教材-《编译原理与技术》张莉等
 - 虎书-《Modern Compiler Implementation in Java》
 - 鲸书-《Advanced Compiler Design and Implementation》
 - 《自制编译器》，(日) 青木峰良

目录

- ① 课程要求与成绩构成
- ② What - 编译程序是什么?
- ③ Why - 我们为什么需要编译程序?
- ④ Why to learn - 为什么要学习编译原理?
- ⑤ 编译系统概述
- ⑥ 总结与作业



写程序的软件



DEV-C++



写程序的软件



DEV-C++



Visual Studio



写程序的软件



DEV-C++



Visual Studio



CLion



写程序的软件



DEV-C++



Visual Studio Code



Visual Studio



CLion



写程序的软件



DEV-C++



Visual Studio



CLion



Visual Studio Code



Vim/NeoVim

写程序的软件



DEV-C++



Visual Studio



CLion



Visual Studio Code



Vim/NeoVim



Emacs

写程序的软件



DEV-C++



Visual Studio Code



Python



Visual Studio



Vim/NeoVim



CLion



Emacs

写程序的软件



DEV-C++



Visual Studio Code



Python



Visual Studio



Vim/NeoVim



GCC



CLion



Emacs

写程序的软件



DEV-C++



Visual Studio Code



Python



Visual Studio



Vim/NeoVim



GCC



CLion



Emacs



Clang/LLVM

编程软件分类

- 集成开发环境 (Integrated Development Environment, **IDE**)



编程软件的分

- 集成开发环境 (Integrated Development Environment, **IDE**)
 - 一键编译、一键运行



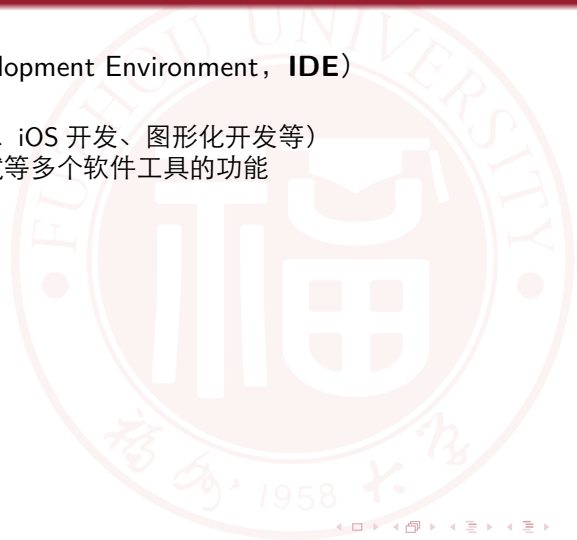
编程软件的分类

- 集成开发环境（Integrated Development Environment, **IDE**）
 - 一键编译、一键运行
 - 复杂项目管理（Android 开发、iOS 开发、图形化开发等）



编程软件的分

- 集成开发环境（Integrated Development Environment, **IDE**）
 - 一键编译、一键运行
 - 复杂项目管理（Android 开发、iOS 开发、图形化开发等）
 - 集合了文本编辑、编译、调试等多个软件工具的功能



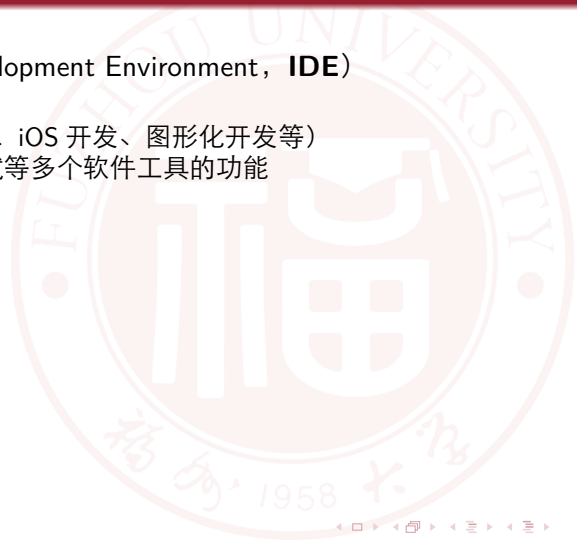
编程软件的分

- 集成开发环境（Integrated Development Environment，**IDE**）
 - 一键编译、一键运行
 - 复杂项目管理（Android 开发、iOS 开发、图形化开发等）
 - 集合了文本编辑、编译、调试等多个软件工具的功能
- 现代文本编辑器



编程软件的分

- 集成开发环境（Integrated Development Environment, **IDE**）
 - 一键编译、一键运行
 - 复杂项目管理（Android 开发、iOS 开发、图形化开发等）
 - 集合了文本编辑、编译、调试等多个软件工具的功能
- 现代文本编辑器
 - 以代码编辑为主要功能



编程软件的分

- 集成开发环境（Integrated Development Environment, **IDE**）
 - 一键编译、一键运行
 - 复杂项目管理（Android 开发、iOS 开发、图形化开发等）
 - 集合了文本编辑、编译、调试等多个软件工具的功能
- 现代文本编辑器
 - 以代码编辑为主要功能
 - 可扩展（JS 脚本、Vim 脚本等）



编程软件的分类

- 集成开发环境（Integrated Development Environment, **IDE**）
 - 一键编译、一键运行
 - 复杂项目管理（Android 开发、iOS 开发、图形化开发等）
 - 集合了文本编辑、编译、调试等多个软件工具的功能
- 现代文本编辑器
 - 以代码编辑为主要功能
 - 可扩展（JS 脚本、Vim 脚本等）
 - 可以根据需求安装各种插件



编程软件的分类

- 集成开发环境（Integrated Development Environment, **IDE**）
 - 一键编译、一键运行
 - 复杂项目管理（Android 开发、iOS 开发、图形化开发等）
 - 集合了文本编辑、编译、调试等多个软件工具的功能
- 现代文本编辑器
 - 以代码编辑为主要功能
 - 可扩展（JS 脚本、Vim 脚本等）
 - 可以根据需求安装各种插件
 - “毛坯房”与“精装房”的区别

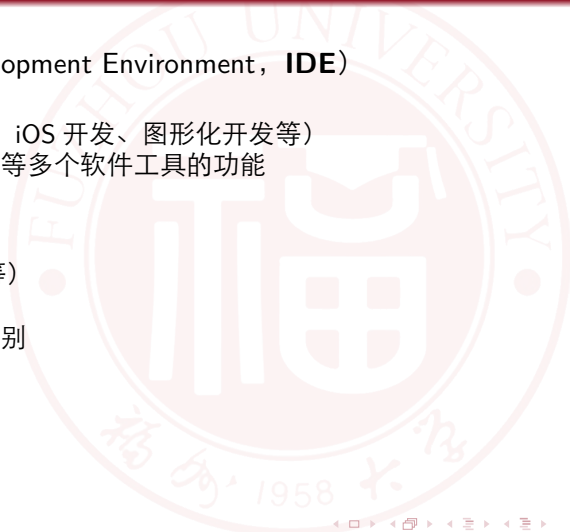


编程软件的分类

- 集成开发环境 (Integrated Development Environment, **IDE**)
 - 一键编译、一键运行
 - 复杂项目管理 (Android 开发、iOS 开发、图形化开发等)
 - 集合了文本编辑、编译、调试等多个软件工具的功能
- 现代文本编辑器
 - 以代码编辑为主要功能
 - 可扩展 (JS 脚本、Vim 脚本等)
 - 可以根据需求安装各种插件
 - “毛坯房”与“精装房”的区别
- 编译程序

编程软件的分

- 集成开发环境（Integrated Development Environment, **IDE**）
 - 一键编译、一键运行
 - 复杂项目管理（Android 开发、iOS 开发、图形化开发等）
 - 集合了文本编辑、编译、调试等多个软件工具的功能
- 现代文本编辑器
 - 以代码编辑为主要功能
 - 可扩展（JS 脚本、Vim 脚本等）
 - 可以根据需求安装各种插件
 - “毛坯房”与“精装房”的区别
- 编译程序
 - 从一种表示到另一种表示



编程软件的分类

- 集成开发环境（Integrated Development Environment, **IDE**）
 - 一键编译、一键运行
 - 复杂项目管理（Android 开发、iOS 开发、图形化开发等）
 - 集合了文本编辑、编译、调试等多个软件工具的功能
- 现代文本编辑器
 - 以代码编辑为主要功能
 - 可扩展（JS 脚本、Vim 脚本等）
 - 可以根据需求安装各种插件
 - “毛坯房”与“精装房”的区别
- 编译程序
 - 从一种表示到另一种表示
 - 直接执行



编程软件的分类

- 集成开发环境 (Integrated Development Environment, **IDE**)
 - 一键编译、一键运行
 - 复杂项目管理 (Android 开发、iOS 开发、图形化开发等)
 - 集合了文本编辑、编译、调试等多个软件工具的功能
- 现代文本编辑器
 - 以代码编辑为主要功能
 - 可扩展 (JS 脚本、Vim 脚本等)
 - 可以根据需求安装各种插件
 - “毛坯房”与“精装房”的区别
- 编译程序
 - 从一种表示到另一种表示
 - 直接执行
 - 解释执行

目录

- ① 课程要求与成绩构成
- ② What - 编译程序是什么?
- ③ Why - 我们为什么需要编译程序?
为什么需要程序设计语言?
程序设计语言的各阶段
- ④ Why to learn - 为什么要学习编译原理?
- ⑤ 编译系统概述



目录

- ① 课程要求与成绩构成
- ② What - 编译程序是什么?
- ③ Why - 我们为什么需要编译程序?
为什么需要程序设计语言?
程序设计语言的各阶段
- ④ Why to learn - 为什么要学习编译原理?
- ⑤ 编译系统概述



为什么需要程序设计语言?

- 语言是传递信息的工具



为什么需要程序设计语言?

- 语言是传递信息的工具
- 人与计算机沟通需要工具



为什么需要程序设计语言?

- 语言是传递信息的工具
- 人与计算机沟通需要工具
- 程序设计语言是人与问题以及协助解决该问题的计算机之间的通信工具

为什么需要程序设计语言?

- 语言是传递信息的工具
- 人与计算机沟通需要工具
- 程序设计语言是人与问题以及协助解决该问题的计算机之间的通信工具
- 程序设计语言是人与计算机之间的中介

帮助解决该问题的计算机之间的通信工具的中介

- 帮助解决该问题的计算机之间的通信工具的中介

帮助解决该问题的计算机之间的通信工具的中介

为什么需要程序设计语言?

- 语言是传递信息的工具
- 人与计算机沟通需要工具
- 程序设计语言是人与问题以及协助解决该问题的计算机之间的通信工具
- 程序设计语言是人与计算机之间的中介

程序设计语言是什么?

- 程序设计语言是一种人工语言，有严格的定义（语法、语义）

为什么需要程序设计语言？

- 语言是传递信息的工具
- 人与计算机沟通需要工具
- 程序设计语言是人与问题以及协助解决该问题的计算机之间的通信工具
- 程序设计语言是人与计算机之间的中介

程序设计语言是什么？

- 程序设计语言是一种人工语言，有严格的定义（语法、语义）
- 优秀的程序设计语言的两个目标：开发效率与表达能力

程序设计语言如何分类？

如何描述一个程序

- 记录计算机执行的每一个步骤——指令式编程

程序设计语言如何分类？

如何描述一个程序

- 记录计算机执行的每一个步骤——指令式编程
- 描述如何从初始数据开始构造目标结果——声明式编程（函数式编程）

程序设计语言如何分类？

如何描述一个程序

- 记录计算机执行的每一个步骤——指令式编程
- 描述如何从初始数据开始构造目标结果——声明式编程（函数式编程）

按照是否与特定机器相关

程序设计语言如何分类？

如何描述一个程序

- 记录计算机执行的每一个步骤——指令式编程
- 描述如何从初始数据开始构造目标结果——声明式编程（函数式编程）

按照是否与特定机器相关

- 与特定机器相关：机器语言、汇编语言

程序设计语言如何分类?

如何描述一个程序

- 记录计算机执行的每一个步骤——指令式编程
- 描述如何从初始数据开始构造目标结果——声明式编程（函数式编程）

按照是否与特定机器相关

- 与特定机器相关：机器语言、汇编语言
- 与特定机器无关：高级语言

目录

- ① 课程要求与成绩构成
- ② What - 编译程序是什么?
- ③ Why - 我们为什么需要编译程序?
为什么需要程序设计语言?
程序设计语言的各阶段
- ④ Why to learn - 为什么要学习编译原理?
- ⑤ 编译系统概述



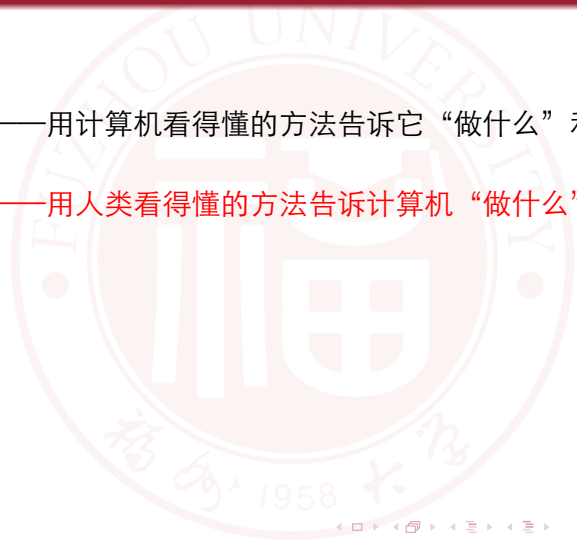
程序设计语言的发展历史

- 第一阶段：机器语言（1940-）——用计算机看得懂的方法告诉它“做什么”和“怎么做”



程序设计语言的发展历史

- 第一阶段：机器语言（1940-）——用计算机看得懂的方法告诉它“做什么”和“怎么做”
- 第二阶段：汇编语言（1950-）——用人类看得懂的方法告诉计算机“做什么”和“怎么做”



程序设计语言的发展历史

- 第一阶段：机器语言（1940-）——用计算机看得懂的方法告诉它“做什么”和“怎么做”
- 第二阶段：汇编语言（1950-）——用人类看得懂的方法告诉计算机“做什么”和“怎么做”
- 第三阶段：通用高级语言（1970-）——用人类的思维告诉计算机“做什么”和“怎么做”

- 用计算机看得懂的方法告诉它“做什么”
- 用人类看得懂的方法告诉计算机“做什么”
- 2) ——用人类的思维告诉计算机“做什么”
- Domain Specific Language, DSL) ——可以只告诉

- 第一阶段：机器语言（1940-）——用计算机看得懂的方法告诉它“做什么”和“怎么做”
- 第二阶段：汇编语言（1950-）——用人类看得懂的方法告诉计算机“做什么”和“怎么做”
- 第三阶段：通用高级语言（1970-）——用人类的思维告诉计算机“做什么”和“怎么做”
- 第四阶段：特定领域语言（Domain Specific Language, DSL）——可以只告诉计算机“做什么”

机器语言

- 通过转换物理开关的状态来设计程序



机器语言

- 通过转换物理开关的状态来设计程序
 - 早期的统计机器：纸张挖孔、探针通电、机械计数



机器语言

- 通过转换物理开关的状态来设计程序
 - 早期的统计机器：纸张挖孔、探针通电、机械计数
 - 打孔纸带：程序和数据通过纸带输入，计算结果通过纸带输出或直接打印。

机器语言

- 通过转换物理开关的状态来设计程序
 - 早期的统计机器：纸张挖孔、探针通电、机械计数
 - 打孔纸带：程序和数据通过纸带输入，计算结果通过纸带输出或直接打印。
 - 现代 CPU：手动设置每一个二进制位来编程

MIPS32 架构指令

	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Type	Op	Rs	Rt	Rd	Shamt	Func
I-Type	Op	Rs	Rt	16 bit Immediate Value or Address Offset		
J-Type	Op	26 bit Address (for Jump Instruction)				

图 1: MIPS32 架构指令结构

MIPS32 架构指令

	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Type	Op	Rs	Rt	Rd	Shamt	Func
I-Type	Op	Rs	Rt	16 bit Immediate Value or Address Offset		
J-Type	Op	26 bit Address (for Jump Instruction)				

图 1: MIPS32 架构指令结构

- R-型指令是寄存器型指令
 - 其操作码 OP 字段是特定的

“000000”，具体操作类型由 func 字段给定。

- 例如：func= “100000” 时，表示“加法”运算。

MIPS32 架构指令

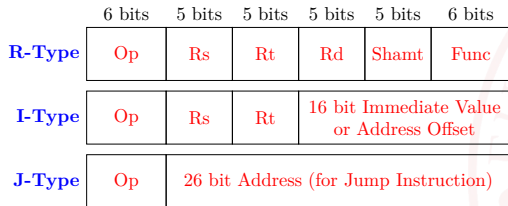


图 1: MIPS32 架构指令结构

- R-型指令是寄存器型指令
 - 其操作码 OP 字段是特定的

“000000”，具体操作类型由 func 字段给定。

- 例如：func= “100000” 时，表示“加法” 运算。
- I-型指令是立即数指令
 - 指令中携带的操作数被称为立即数
 - MIPS32 支持的立即数固定为 16 位

MIPS32 架构指令

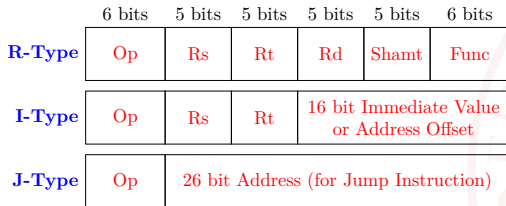


图 1: MIPS32 架构指令结构

- R-型指令是寄存器型指令
 - 其操作码 OP 字段是特定的

“000000”，具体操作类型由 func 字段给定。

- 例如：func= “100000” 时，表示“加法”运算。
- I-型指令是立即数指令
 - 指令中携带的操作数被称为立即数
 - MIPS32 支持的立即数固定为 16 位
- J-型指令主要是无条件跳转指令，将当前 PC 的高 4 位拼上 26 位立即数，后补两个“0”，作为跳转目标地址。

MIPS32 机器编程示例

	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Type	Op	Rs	Rt	Rd	Shamt	Func
I-Type	Op	Rs	Rt	16 bit Immediate Value or Address Offset		
J-Type	Op	26 bit Address (for Jump Instruction)				

● 这是一条 I-型指令

一条二进制 MIPS 指令为

00100100000010000000000000011110

具体解释如下：

MIPS32 机器编程示例

	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Type	Op	Rs	Rt	Rd	Shamt	Func
I-Type	Op	Rs	Rt	16 bit Immediate Value or Address Offset		
J-Type	Op	26 bit Address (for Jump Instruction)				

- 这是一条 I-型指令
- 操作码为 001001，查表为 addiu 指令——立即数无符号加法

一条二进制 MIPS 指令为

001001000000100000000000000011110

具体解释如下：

MIPS32 机器编程示例

	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Type	Op	Rs	Rt	Rd	Shamt	Func
I-Type	Op	Rs	Rt	16 bit Immediate Value or Address Offset		
J-Type	Op	26 bit Address (for Jump Instruction)				

- 这是一条 I-型指令
- 操作码为 001001，查表为 addiu 指令——立即数无符号加法
- Rs 为 00000，即零寄存器

一条二进制 MIPS 指令为

001001000000100000000000000011110

具体解释如下：

MIPS32 机器编程示例

	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Type	Op	Rs	Rt	Rd	Shamt	Func
I-Type	Op	Rs	Rt	16 bit Immediate Value or Address Offset		
J-Type	Op	26 bit Address (for Jump Instruction)				

- 这是一条 I-型指令
- 操作码为 001001，查表为 addiu 指令——立即数无符号加法
- Rs 为 00000，即零寄存器
- Rt 为 01000，8 号寄存器，第一个用于存储临时值的寄存器

一条二进制 MIPS 指令为

0010010000001000000000000011110

具体解释如下：

MIPS32 机器编程示例

	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Type	Op	Rs	Rt	Rd	Shamt	Func
I-Type	Op	Rs	Rt	16 bit Immediate Value or Address Offset		
J-Type	Op	26 bit Address (for Jump Instruction)				

一条二进制 MIPS 指令为

0010010000001000000000000011110

具体解释如下：

- 这是一条 I-型指令
- 操作码为 001001，查表为 addiu 指令——立即数无符号加法
- Rs 为 00000，即零寄存器
- Rt 为 01000，8 号寄存器，第一个用于存储临时值的寄存器
- 指令的立即数为 00000000000011110，十进制为 30

MIPS32 机器编程示例

	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
R-Type	Op	Rs	Rt	Rd	Shamt	Func
I-Type	Op	Rs	Rt	16 bit Immediate Value or Address Offset		
J-Type	Op	26 bit Address (for Jump Instruction)				

一条二进制 MIPS 指令为

0010010000001000000000000011110

具体解释如下：

- 这是一条 I-型指令
- 操作码为 001001，查表为 addiu 指令——立即数无符号加法
- Rs 为 00000，即零寄存器
- Rt 为 01000，8 号寄存器，第一个用于存储临时值的寄存器
- 指令的立即数为 00000000000011110，十进制为 30
- 作用是把 0+30 的结果存到 8 号寄存器中

机器语言编程

- MIPS 指令集总共包含约 111 条指令，每条指令以 32 位表示。



机器语言编程

- MIPS 指令集总共包含约 111 条指令，每条指令以 32 位表示。
- MIPS32 指令手册



机器语言编程

- MIPS 指令集总共包含约 111 条指令，每条指令以 32 位表示。
- MIPS32 指令手册
- 请用 MIPS 指令编写 `print(30+20)`



机器语言编程

- MIPS 指令集总共包含约 111 条指令，每条指令以 32 位表示。
- MIPS32 指令手册
- 请用 MIPS 指令编写 `print(30+20)`

机器指令如下：



机器语言编程

- MIPS 指令集总共包含约 111 条指令，每条指令以 32 位表示。
- MIPS32 指令手册
- 请用 MIPS 指令编写 `print(30+20)`

机器指令如下：

- 0010010000000100000000000000011110

机器语言编程

- MIPS 指令集总共包含约 111 条指令，每条指令以 32 位表示。
- MIPS32 指令手册
- 请用 MIPS 指令编写 `print(30+20)`

机器指令如下：

- 0010010000000100000000000000011110
- 0010010000000100100000000000010100

机器语言编程

- MIPS 指令集总共包含约 111 条指令，每条指令以 32 位表示。
- MIPS32 指令手册
- 请用 MIPS 指令编写 `print(30+20)`

机器指令如下：

- 0010010000000100000000000000011110
- 0010010000000100100000000000010100
- 0000000010000100100100000000100000

机器语言编程

- MIPS 指令集总共包含约 111 条指令，每条指令以 32 位表示。
- MIPS32 指令手册
- 请用 MIPS 指令编写 `print(30+20)`

机器指令如下：

- 001001000000100000000000000011110
- 001001000000100100000000000010100
- 00000001000010010010000000100000
- 00100100000000100000000000000001

机器语言编程

- MIPS 指令集总共包含约 111 条指令，每条指令以 32 位表示。
- MIPS32 指令手册
- 请用 MIPS 指令编写 `print(30+20)`

机器指令如下：

- 001001000000100000000000000011110
- 001001000000100100000000000010100
- 00000001000010010010000000100000
- 00100100000000100000000000000001
- 0000000000000000000000000000001100

机器语言编程

- 写完二进制就结束了吗？



机器语言编程

- 写完二进制就结束了吗？
 - 如何存储？



机器语言编程

- 写完二进制就结束了吗？
 - 如何存储？
- 直接存储 01 字符？



机器语言编程

- 写完二进制就结束了吗？
 - 如何存储？
- 直接存储 01 字符？
 - 太浪费空间了



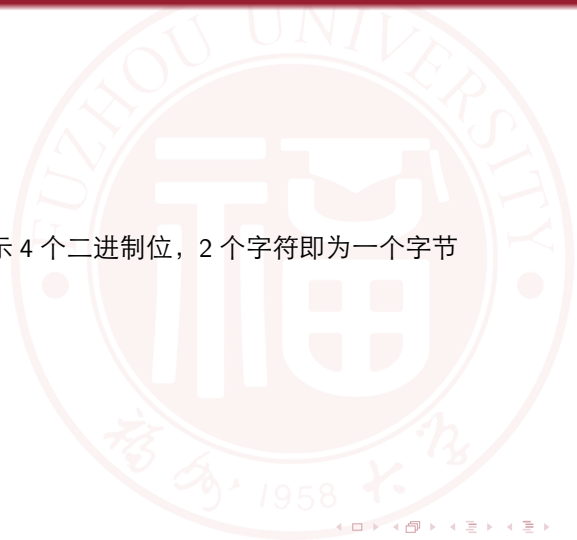
机器语言编程

- 写完二进制就结束了吗？
 - 如何存储？
- 直接存储 01 字符？
 - 太浪费空间了
- 按字节紧凑存储



机器语言编程

- 写完二进制就结束了吗？
 - 如何存储？
- 直接存储 01 字符？
 - 太浪费空间了
- 按字节紧凑存储
 - 1 个十六进制（Hex）字符表示 4 个二进制位，2 个字符即为一个字节



示 4 个二进制位，2 个字符即为一个字节

	Hex	字节 1	Hex	字节 2	Hex	字节 3	Hex
00	24	00001000	08	00000000	00	00011110	1E
00	24	00001001	09	00000000	00	00010100	14
01	01	00001001	09	00100000	20	00100000	20
00	24	00000010	02	00000000	00	00000001	01
00	00	00000000	00	00000000	00	00001100	0C

- 示 4 个二进制位，2 个字符即为一个字节
- | | Hex | 字节 1 | Hex | 字节 2 | Hex | 字节 3 | Hex |
|----|-----|----------|-----|----------|-----|----------|-----|
| 00 | 24 | 00001000 | 08 | 00000000 | 00 | 00011110 | 1E |
| 00 | 24 | 00001001 | 09 | 00000000 | 00 | 00010100 | 14 |
| 01 | 01 | 00001001 | 09 | 00100000 | 20 | 00100000 | 20 |
| 00 | 24 | 00000010 | 02 | 00000000 | 00 | 00000001 | 01 |
| 00 | 00 | 00000000 | 00 | 00000000 | 00 | 00001100 | 0C |

示 4 个二进制位，2 个字符即为一个字节

	Hex	字节 1	Hex	字节 2	Hex	字节 3	Hex
00	24	00001000	08	00000000	00	00011110	1E
00	24	00001001	09	00000000	00	00010100	14
01	01	00001001	09	00100000	20	00100000	20
00	24	00000010	02	00000000	00	00000001	01
00	00	00000000	00	00000000	00	00001100	0C

机器语言编程

- 写完二进制就结束了吗？
 - 如何存储？
- 直接存储 01 字符？
 - 太浪费空间了
- 按字节紧凑存储
 - 1 个十六进制 (Hex) 字符表示 4 个二进制位，2 个字符即为一个字节

原始指令	Hex
00100100000010000000000000011110	2408001e
00100100000010010000000000010100	24090014
000000010000100100100000000100000	01092020
001001000000000100000000000000001	24020001
00000000000000000000000000000001100	0000000c

机器语言的优缺点

优点

- 完全的指令式编程



机器语言的优缺点

优点

- 完全的指令式编程
- 机器可以直接执行



机器语言的优缺点

优点

- 完全的指令式编程
- 机器可以直接执行

缺点

机器语言的优缺点

优点

- 完全的指令式编程
- 机器可以直接执行

缺点

- 不够直观

机器语言的优缺点

优点

- 完全的指令式编程
- 机器可以直接执行

缺点

- 不够直观
- 缺少抽象

机器语言的优缺点

优点

- 完全的指令式编程
- 机器可以直接执行

缺点

- 不够直观
- 缺少抽象
- 对程序员要求过高，需要记忆和管理的资源太多了

机器语言的优缺点

优点

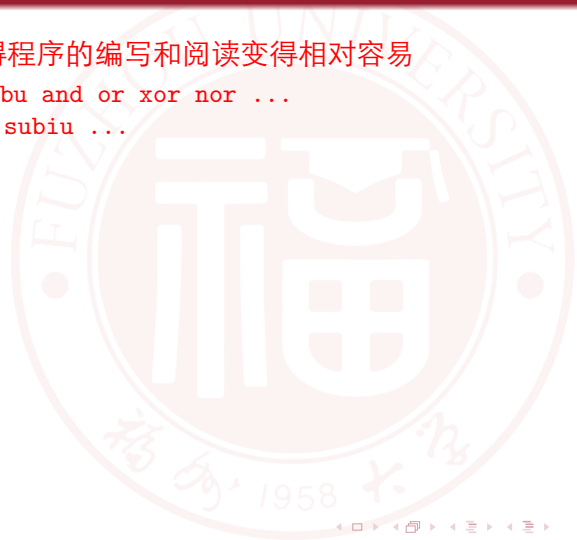
- 完全的指令式编程
- 机器可以直接执行

缺点

- 不够直观
- 缺少抽象
- 对程序员要求过高，需要记忆和管理的资源太多了
- 翻译过程繁琐易错

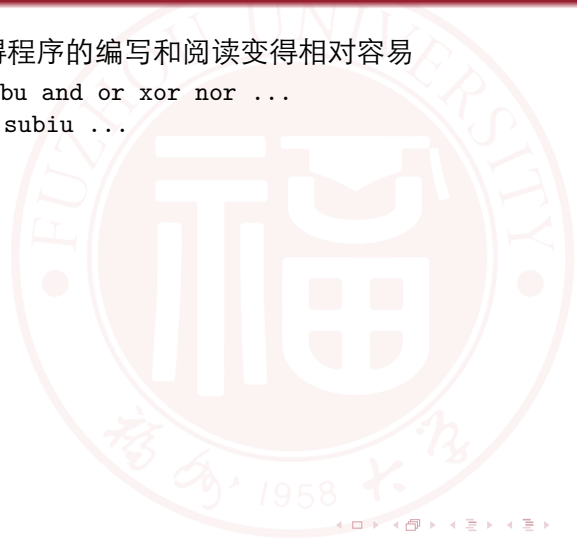
汇编语言——程序设计语言的第二个阶段

- 用助记符来表示机器指令，使得程序的编写和阅读变得相对容易
 - R-型指令: `add addu sub subu and or xor nor ...`
 - I-型指令: `addi addiu subi subiu ...`
 - J-型指令: `j jal ...`



汇编语言——程序设计语言的第二个阶段

- 用助记符来表示机器指令，使得程序的编写和阅读变得相对容易
 - R-型指令: `add addu sub subu and or xor nor ...`
 - I-型指令: `addi addiu subi subiu ...`
 - J-型指令: `j jal ...`
- 与机器语言基本 “一一” 对应



汇编语言——程序设计语言的第二个阶段

- 用助记符来表示机器指令，使得程序的编写和阅读变得相对容易
 - R-型指令: `add addu sub subu and or xor nor ...`
 - I-型指令: `addi addiu subi subiu ...`
 - J-型指令: `j jal ...`
- 与机器语言基本 “——” 对应
- 通过汇编器 (**Assembler**) 将其转换为二进制机器代码 (**Binary Code, bin**)

汇编语言——程序设计语言的第二个阶段

- 用助记符来表示机器指令，使得程序的编写和阅读变得相对容易
 - R-型指令: `add addu sub subu and or xor nor ...`
 - I-型指令: `addi addiu subi subiu ...`
 - J-型指令: `j jal ...`
- 与机器语言基本 “——” 对应
- 通过**汇编器 (Assembler)** 将其转换为二进制机器代码 (**Binary Code, bin**)
- 汇编器会帮忙计算跳转位置 (通过标签来记录位置) 和处理一些复杂操作 (如导入大整数、超范围跳转)¹

汇编语言——程序设计语言的第二个阶段

- 用助记符来表示机器指令，使得程序的编写和阅读变得相对容易
 - R-型指令: `add addu sub subu and or xor nor ...`
 - I-型指令: `addi addiu subi subiu ...`
 - J-型指令: `j jal ...`
- 与机器语言基本 “——” 对应
- 通过**汇编器 (Assembler)** 将其转换为二进制机器代码 (**Binary Code, bin**)
- 汇编器会帮忙计算跳转位置 (通过标签来记录位置) 和处理一些复杂操作 (如导入大整数、超范围跳转)¹
 - 汇编指令 `la $a0, str`, 将 `str` 标签的地址加载到寄存器 `a0` 中。翻译为二进制后为多条指令:
 - `addiu $a0, $zero, str[hi]: 0010010000000010000000000011000000`
 - `sll $a0, $a0, 16: 0000000000000010000100100000000000`
 - `addiu $a0, $a0, str[lo]: 001001000000000100000000000000100`

汇编语言——程序设计语言的第二个阶段

- 用助记符来表示机器指令，使得程序的编写和阅读变得相对容易
 - R-型指令: `add addu sub subu and or xor nor ...`
 - I-型指令: `addi addiu subi subiu ...`
 - J-型指令: `j jal ...`
- 与机器语言基本 “——” 对应
- 通过**汇编器 (Assembler)** 将其转换为二进制机器代码 (**Binary Code, bin**)
- 汇编器会帮忙计算跳转位置（通过标签来记录位置）和处理一些复杂操作（如导入大整数、超范围跳转）¹
 - 汇编指令 `la $a0, str`，将 `str` 标签的地址加载到寄存器 `a0` 中。翻译为二进制后为多条指令：
 - `addiu $a0, $zero, str[hi]: 0010010000000010000000000011000000`
 - `sll $a0, $a0, 16: 0000000000000010000100100000000000`
 - `addiu $a0, $a0, str[lo]: 001001000000000100000000000000100`

汇编语言——程序设计语言的第二个阶段

- 用助记符来表示机器指令，使得程序的编写和阅读变得相对容易
 - R-型指令: `add addu sub subu and or xor nor ...`
 - I-型指令: `addi addiu subi subiu ...`
 - J-型指令: `j jal ...`
- 与机器语言基本 “——” 对应
- 通过**汇编器 (Assembler)** 将其转换为二进制机器代码 (**Binary Code, bin**)
- 汇编器会帮忙计算跳转位置 (通过标签来记录位置) 和处理一些复杂操作 (如导入大整数、超范围跳转)¹
 - 汇编指令 `la $a0, str`, 将 `str` 标签的地址加载到寄存器 `a0` 中。翻译为二进制后为多条指令:
 - `addiu $a0, $zero, str[hi]: 0010010000000010000000000011000000`
 - `sll $a0, $a0, 16: 0000000000000010000100100000000000`
 - `addiu $a0, $a0, str[lo]: 0010010000000001000000000000000100`

¹ 机器代码通过在线汇编器生成: <https://www.csfieldguide.org.nz/en/interactives/mips-assembler/>

意大利面条式编程——催生了重要的编程范式

- 汇编语言的抽象层次低，需要直接控制硬件资源



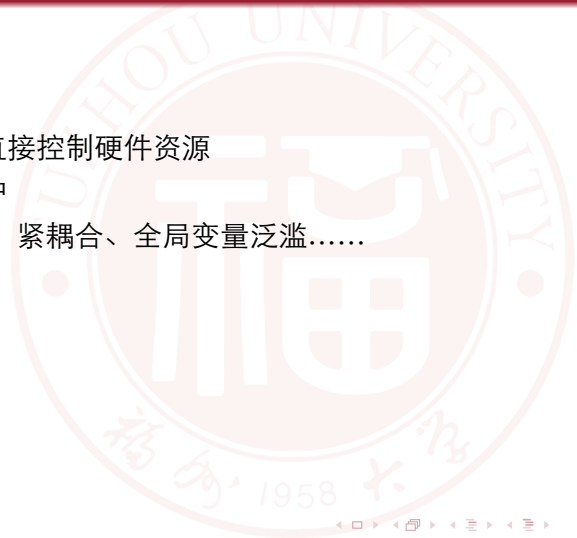
意大利面条式编程——催生了重要的编程范式

- 汇编语言的抽象层次低，需要直接控制硬件资源
- 大多数程序结构混乱，难以维护



意大利面条式编程——催生了重要的编程范式

- 汇编语言的抽象层次低，需要直接控制硬件资源
- 大多数程序结构混乱，难以维护
- 滥用 GOTO 语句、缺乏模块化、紧耦合、全局变量泛滥.....



意大利面条式编程——催生了重要的编程范式

- 汇编语言的抽象层次低，需要直接控制硬件资源
- 大多数程序结构混乱，难以维护
- 滥用 GOTO 语句、缺乏模块化、紧耦合、全局变量泛滥.....
- 被称为意大利面条式编程，形容代码像一碗面条一样，相互纠缠、扭结在一起，理不清头绪

意大利面条式编程——催生了重要的编程范式

- 汇编语言的抽象层次低，需要直接控制硬件资源
- 大多数程序结构混乱，难以维护
- 滥用 GOTO 语句、缺乏模块化、紧耦合、全局变量泛滥.....
- 被称为意大利面条式编程，形容代码像一碗面条一样，相互纠缠、扭结在一起，理不清头绪
- 促成了现代软件工程思想、方法和最佳实践的发展

高级语言——抽象化的飞跃

- 汇编语言的复杂性推动了对更高层次抽象的需求



高级语言——抽象化的飞跃

- 汇编语言的复杂性推动了对更高层次抽象的需求
- 高级语言引入了变量、数据类型、控制结构、函数等概念



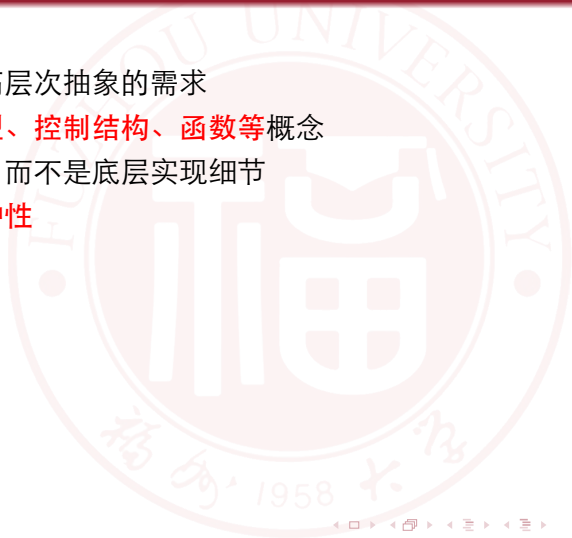
高级语言——抽象化的飞跃

- 汇编语言的复杂性推动了对更高层次抽象的需求
- 高级语言引入了变量、数据类型、控制结构、函数等概念
- 使程序员能够专注于解决问题，而不是底层实现细节



高级语言——抽象化的飞跃

- 汇编语言的复杂性推动了对更高层次抽象的需求
- 高级语言引入了变量、数据类型、控制结构、函数等概念
- 使程序员能够专注于解决问题，而不是底层实现细节
- 促进了软件开发的效率和可维护性



高级语言——抽象化的飞跃

- 汇编语言的复杂性推动了对更高层次抽象的需求
- 高级语言引入了变量、数据类型、控制结构、函数等概念
- 使程序员能够专注于解决问题，而不是底层实现细节
- 促进了软件开发的效率和可维护性
- 现代高级语言（如 C、C++、Java、Python 等）提供了丰富的库和框架，进一步简化了开发过程

高级语言——抽象化的飞跃

- 汇编语言的复杂性推动了对更高层次抽象的需求
- 高级语言引入了**变量、数据类型、控制结构、函数等**概念
- 使程序员能够专注于解决问题，而不是底层实现细节
- 促进了软件开发的**效率**和**可维护性**
- 现代高级语言（如 C、C++、Java、Python 等）提供了**丰富的库和框架**，进一步**简化了开发过程**
- 开发者**无需深入理解 CPU 架构、寄存器等底层细节，无需修改代码便可通过编译器或解释器适配不同平台（如 x86、ARM）**

高级语言——抽象化的飞跃

- 汇编语言的复杂性推动了对更高层次抽象的需求
- 高级语言引入了**变量、数据类型、控制结构、函数等**概念
- 使程序员能够专注于解决问题，而不是底层实现细节
- 促进了软件开发的**效率**和**可维护性**
- 现代高级语言（如 C、C++、Java、Python 等）提供了**丰富的库和框架**，进一步**简化了开发过程**
- 开发者**无需**深入理解 CPU 架构、寄存器等底层细节，**无需**修改代码便可通过编译器或解释器适配不同平台（如 x86、ARM）
- **可读性与可维护性高**：语法更接近人类语言，结构清晰，便于理解和协作；支持模块化、面向对象等设计，便于代码维护和扩展

甚高级语言——特定领域语言（DSL）的兴起

- 高级语言的成功催生了对更高层次抽象的需求



甚高级语言——特定领域语言（DSL）的兴起

- 高级语言的成功催生了对更高层次抽象的需求
- 特定领域语言（Domain Specific Language, DSL）专注于特定应用领域



甚高级语言——特定领域语言（DSL）的兴起

- 高级语言的成功催生了对更高层次抽象的需求
- 特定领域语言（Domain Specific Language, DSL）专注于特定应用领域
- 例如：SQL 用于数据库查询，HTML 用于网页设计，MATLAB 用于数值计算



甚高级语言——特定领域语言（DSL）的兴起

- 高级语言的成功催生了对更高层次抽象的需求
- 特定领域语言（Domain Specific Language, DSL）专注于特定应用领域
- 例如：SQL 用于数据库查询，HTML 用于网页设计，MATLAB 用于数值计算
- DSL 通常只描述结果而不考虑中间过程式怎么样的

甚高级语言——特定领域语言（DSL）的兴起

- 高级语言的成功催生了对更高层次抽象的需求
- 特定领域语言（Domain Specific Language, DSL）专注于特定应用领域
- 例如：SQL 用于数据库查询，HTML 用于网页设计，MATLAB 用于数值计算
- DSL 通常只描述结果而不考虑中间过程式怎么样的
- 通常通过解释器或编译器将其转换为通用高级语言或机器代码

思考题

Unix 系统是 C 语言编写的，二者都是美国贝尔实验室的研究成果。那么是先有 Unix 操作系统还是先有 C 语言？



思考题

Unix 系统是 C 语言编写的，二者都是美国贝尔实验室的研究成果。那么是先有 Unix 操作系统还是先有 C 语言？

答

Unix 操作系统先于 C 语言诞生。最初，Unix 是用汇编语言编写的。后来，为了提高可移植性和开发效率，Dennis Ritchie 在 Ken Thompson 的协助下设计了 C 语言，并将 Unix 操作系统的大部分代码重写为 C 语言。这一转变使得 Unix 操作系统能够更容易地移植到不同的硬件平台上。

C 语言的可高效移植的特性推动了 Unix 操作系统的发展和普及。Unix 系统的普及也促进了 C 语言的广泛应用，二者相辅相成，共同推动了计算机科学的发展。

目录

- ① 课程要求与成绩构成
- ② What - 编译程序是什么？
- ③ Why - 我们为什么需要编译程序？
- ④ Why to learn - 为什么要学习编译原理？
- ⑤ 编译系统概述
- ⑥ 总结与作业



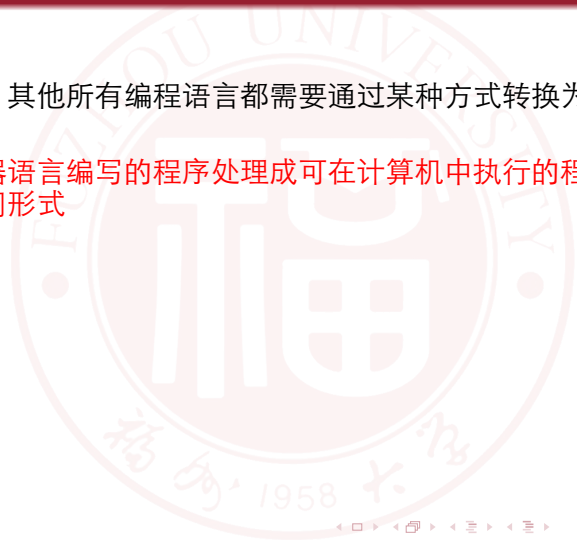
为什么需要编译程序？

- 除了机器语言可以直接执行外，其他所有编程语言都需要通过某种方式转换为机器语言，才能被计算机执行



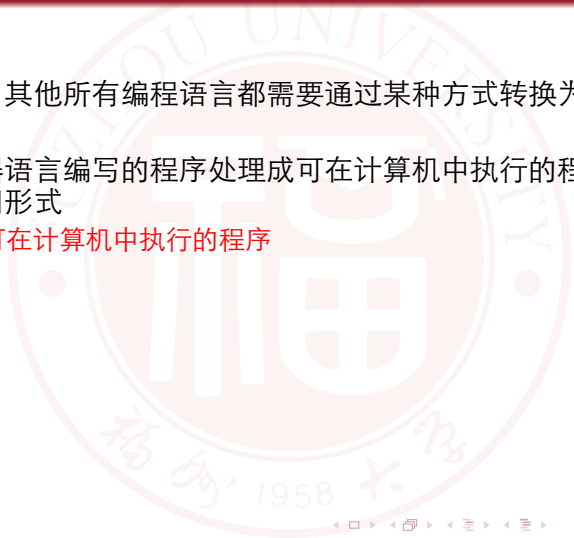
为什么需要编译程序?

- 除了机器语言可以直接执行外，其他所有编程语言都需要通过某种方式转换为机器语言，才能被计算机执行
- 语言处理系统的作用是将非机器语言编写的程序处理成可在计算机中执行的程序、最终执行的结果或其他中间形式



为什么需要编译程序？

- 除了机器语言可以直接执行外，其他所有编程语言都需要通过某种方式转换为机器语言，才能被计算机执行
- 语言处理系统的作用是将非机器语言编写的程序处理成可在计算机中执行的程序、最终执行的结果或其他中间形式
 - 编译器 (Compiler)：处理成可在计算机中执行的程序



为什么需要编译程序?

- 除了机器语言可以直接执行外，其他所有编程语言都需要通过某种方式转换为机器语言，才能被计算机执行
- 语言处理系统的作用是将非机器语言编写的程序处理成可在计算机中执行的程序、最终执行的结果或其他中间形式
 - 编译器 (Compiler): 处理成可在计算机中执行的程序
 - 解释器 (Interpreter): 处理成最终执行的结果

为什么需要编译程序？

- 除了机器语言可以直接执行外，其他所有编程语言都需要通过某种方式转换为机器语言，才能被计算机执行
- 语言处理系统的作用是将非机器语言编写的程序处理成可在计算机中执行的程序、最终执行的结果或其他中间形式
 - 编译器 (Compiler)：处理成可在计算机中执行的程序
 - 解释器 (Interpreter)：处理成最终执行的结果
 - 混合型：先编译后解释，如 Java、Python 等

为什么需要编译程序？

- 除了机器语言可以直接执行外，其他所有编程语言都需要通过某种方式转换为机器语言，才能被计算机执行
- 语言处理系统的作用是将非机器语言编写的程序处理成可在计算机中执行的程序、最终执行的结果或其他中间形式
 - 编译器 (Compiler)：处理成可在计算机中执行的程序
 - 解释器 (Interpreter)：处理成最终执行的结果
 - 混合型：先编译后解释，如 Java、Python 等
- 提高计算机程序的开发效率、可读性和可维护性

为什么需要编译程序？

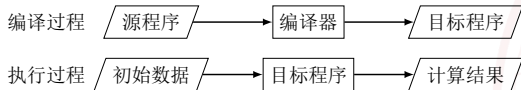
- 除了机器语言可以直接执行外，其他所有编程语言都需要通过某种方式转换为机器语言，才能被计算机执行
- 语言处理系统的作用是将非机器语言编写的程序处理成可在计算机中执行的程序、最终执行的结果或其他中间形式
 - 编译器 (Compiler)：处理成可在计算机中执行的程序
 - 解释器 (Interpreter)：处理成最终执行的结果
 - 混合型：先编译后解释，如 Java、Python 等
- 提高计算机程序的开发效率、可读性和可维护性
- 降低计算机程序在不同硬件平台之间的迁移成本

为什么需要编译程序？

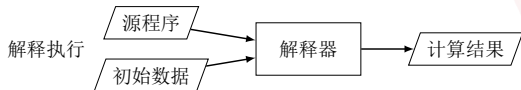
- 除了机器语言可以直接执行外，其他所有编程语言都需要通过某种方式转换为机器语言，才能被计算机执行
- 语言处理系统的作用是将非机器语言编写的程序处理成可在计算机中执行的程序、最终执行的结果或其他中间形式
 - 编译器 (Compiler)：处理成可在计算机中执行的程序
 - 解释器 (Interpreter)：处理成最终执行的结果
 - 混合型：先编译后解释，如 Java、Python 等
- 提高计算机程序的开发效率、可读性和可维护性
- 降低计算机程序在不同硬件平台之间的迁移成本
- 优秀的编译程序可以显著提升程序的执行效率

编译、解释与混合型语言

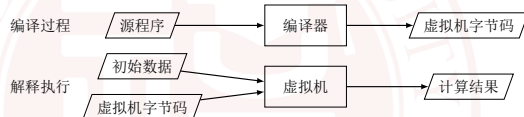
编译型语言



解释型语言

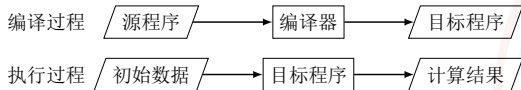


混合型语言

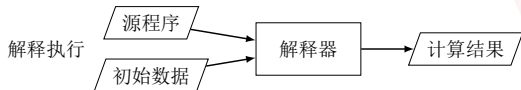


编译、解释与混合型语言

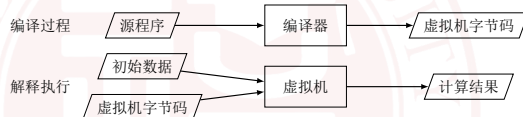
编译型语言



解释型语言



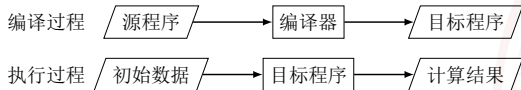
混合型语言



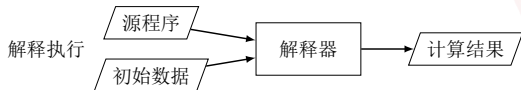
- Java: Java 源代码 → 字节码（编译器）→ JVM 解释执行

编译、解释与混合型语言

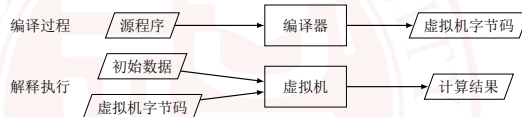
编译型语言



解释型语言



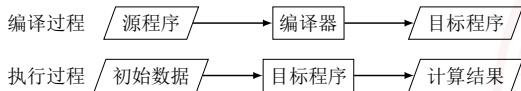
混合型语言



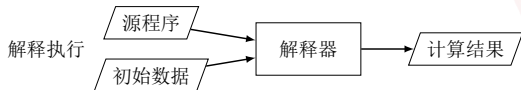
- Java: Java 源代码 → 字节码（编译器）→ JVM 解释执行
- Python: Python 源代码 → 字节码（编译器）→ Python 虚拟机解释执行

编译、解释与混合型语言

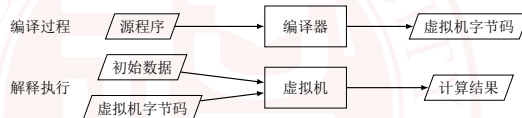
编译型语言



解释型语言



混合型语言



- Java: Java 源代码 → 字节码（编译器）→ JVM 解释执行
- Python: Python 源代码 → 字节码（编译器）→ Python 虚拟机解释执行
- 现在几乎没有纯解释型语言了

为什么要学习编译原理?

国家战略层面

- 信息安全的基石



为什么要学习编译原理？

国家战略层面

- 信息安全的基石
 - 肯·汤普森的万能登录权限



为什么要学习编译原理？

国家战略层面

- 信息安全的基石
 - 肯·汤普森的万能登录权限
 - XcodeGhost 事件



为什么要学习编译原理？

国家战略层面

- 信息安全的基石
 - 肯·汤普森的万能登录权限
 - XcodeGhost 事件
 - 在编译器、操作系统等底层工具中需严格防范供应链攻击。

为什么要学习编译原理?

国家战略层面

- 信息安全的基石
 - 肯·汤普森的万能登录权限
 - XcodeGhost 事件
 - 在编译器、操作系统等底层工具中需严格防范供应链攻击。
- 自主技术生态构建的基石

为什么要学习编译原理?

国家战略层面

- 信息安全的基石
 - 肯·汤普森的万能登录权限
 - XcodeGhost 事件
 - 在编译器、操作系统等底层工具中需严格防范供应链攻击。
- 自主技术生态构建的基石
 - 自主研发的 LoongArch、MIPS、ARM、RISC-V 等新硬件产品需要对应的软件生态

为什么要学习编译原理?

国家战略层面

- 信息安全的基石
 - 肯·汤普森的万能登录权限
 - XcodeGhost 事件
 - 在编译器、操作系统等底层工具中需严格防范供应链攻击。
- 自主技术生态构建的基石
 - 自主研发的 LoongArch、MIPS、ARM、RISC-V 等新硬件产品需要对应的软件生态
 - 软件的平台相关优化是必要的，需要编译程序的支持

为什么要学习编译原理?

国家战略层面

- 信息安全的基石
 - 肯·汤普森的万能登录权限
 - XcodeGhost 事件
 - 在编译器、操作系统等底层工具中需严格防范供应链攻击。
- 自主技术生态构建的基石
 - 自主研发的 LoongArch、MIPS、ARM、RISC-V 等新硬件产品需要对应的软件生态
 - 软件的平台相关优化是必要的，需要编译程序的支持
 - 通过编译技术对 AI 框架进行底层指令级优化，可显著提升模型训练和推理效率，减少对高端芯片的依赖

为什么要学习编译原理？（续）

个人发展层面

- 提升对计算机系统的本质理解
 - 理解变量、函数、内存管理等概念在底层的实现逻辑
 - 掌握编译器如何处理代码，能更高效地调试和优化程序



为什么要学习编译原理？（续）

个人发展层面

- 提升对计算机系统的本质理解
 - 理解变量、函数、内存管理等概念在底层的实现逻辑
 - 掌握编译器如何处理代码，能更高效地调试和优化程序
- 培养复杂问题的解决能力
 - 分层抽象与模块化设计、算法与数据结构的重要实战

为什么要学习编译原理？（续）

个人发展层面

- 提升对计算机系统的本质理解
 - 理解变量、函数、内存管理等概念在底层的实现逻辑
 - 掌握编译器如何处理代码，能更高效地调试和优化程序
- 培养复杂问题的解决能力
 - 分层抽象与模块化设计、算法与数据结构的重要实战
- 拓展技术视野与职业竞争力
 - 自定义工具链开发、深入前沿技术领域

为什么要学习编译原理？（续）

个人发展层面

- 提升对计算机系统的本质理解
 - 理解变量、函数、内存管理等概念在底层的实现逻辑
 - 掌握编译器如何处理代码，能更高效地调试和优化程序
- 培养复杂问题的解决能力
 - 分层抽象与模块化设计、算法与数据结构的重要实战
- 拓展技术视野与职业竞争力
 - 自定义工具链开发、深入前沿技术领域
- 增强代码质量与安全意识
 - 代码优化能力、漏洞防范

为什么要学习编译原理？（续）

个人发展层面

- 提升对计算机系统的本质理解
 - 理解变量、函数、内存管理等概念在底层的实现逻辑
 - 掌握编译器如何处理代码，能更高效地调试和优化程序
- 培养复杂问题的解决能力
 - 分层抽象与模块化设计、算法与数据结构的重要实战
- 拓展技术视野与职业竞争力
 - 自定义工具链开发、深入前沿技术领域
- 增强代码质量与安全意识
 - 代码优化能力、漏洞防范
- 突破职业发展瓶颈
 - 差异化竞争力、技术领导力的基础

目录

- ① 课程要求与成绩构成
- ② What - 编译程序是什么？
- ③ Why - 我们为什么需要编译程序？
- ④ Why to learn - 为什么要学习编译原理？
- ⑤ 编译系统概述
- ⑥ 总结与作业



编译系统

翻译程序（translator）将某种语言编写的程序转换为另一种语言的程序的统称



编译系统

翻译程序（translator）将某种语言编写的程序转换为另一种语言的程序的统称
编译器（compiler）也称编译程序，是一种将源程序转换为目标程序的翻译程序



编译系统

翻译程序 (translator) 将某种语言编写的程序转换为另一种语言的程序的统称
 编译器 (compiler) 也称编译程序，是一种将源程序转换为目标程序的翻译程序
 编译系统 (compilation system) 现代计算机系统中，编译器通常极度复杂，包含多个阶段，涉及到数个不同的程序，故称为编译系统

编译系统

翻译程序 (translator) 将某种语言编写的程序转换为另一种语言的程序的统称
 编译器 (compiler) 也称编译程序，是一种将源程序转换为目标程序的翻译程序
 编译系统 (compilation system) 现代计算机系统中，编译器通常极度复杂，包含多个阶段，涉及到数个不同的程序，故称为编译系统
 宿主语言 (host language) 编译器本身所使用的编程语言

编译系统

翻译程序 (translator) 将某种语言编写的程序转换为另一种语言的程序的统称
 编译器 (compiler) 也称编译程序，是一种将源程序转换为目标程序的翻译程序
 编译系统 (compilation system) 现代计算机系统中，编译器通常极度复杂，包含多个阶段，涉及到数个不同的程序，故称为编译系统
 宿主语言 (host language) 编译器本身所使用的编程语言
 宿主机 (host machine) 运行编译器计算机系统

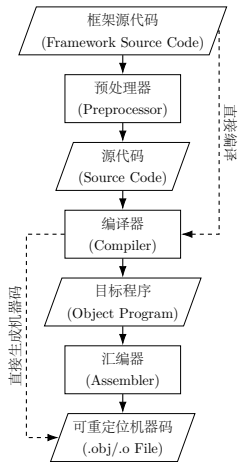
编译系统

翻译程序 (translator) 将某种语言编写的程序转换为另一种语言的程序的统称
 编译器 (compiler) 也称编译程序，是一种将源程序转换为目标程序的翻译程序
 编译系统 (compilation system) 现代计算机系统中，编译器通常极度复杂，包含多个阶段，涉及到数个不同的程序，故称为编译系统
 宿主语言 (host language) 编译器本身所使用的编程语言
 宿主机 (host machine) 运行编译器计算机系统
 自举 (bootstrapping) 使用某种语言编写的编译器在该语言的编译系统上进行编译的过程

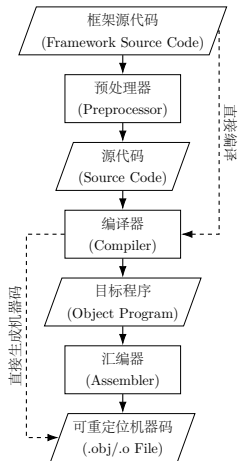
编译系统的构成

单个文件的编译流程

- 预处理：处理文件导入、宏定义等



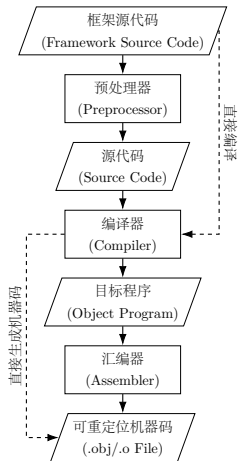
编译系统的构成



单个文件的编译流程

- 预处理：处理文件导入、宏定义等
- 编译：将源代码转换为目标代码，通常为目标的汇编指令

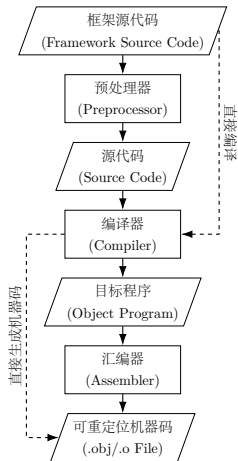
编译系统的构成



单个文件的编译流程

- 预处理：处理文件导入、宏定义等
- 编译：将源代码转换为目标代码，通常为平台目标的汇编指令
- 汇编：将汇编指令转换为机器代码，输出为可重定位的目标文件（Relocatable Object File）

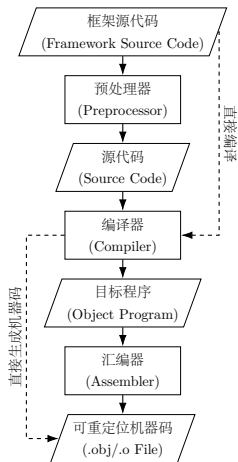
编译系统的构成



单个文件的编译流程

- 预处理：处理文件导入、宏定义等
- 编译：将源代码转换为目标代码，通常为平台目标的汇编指令
- 汇编：将汇编指令转换为机器代码，输出为可重定位的目标文件（Relocatable Object File）
- 对于某些语言或者部分文件，可以跳过预处理和汇编阶段

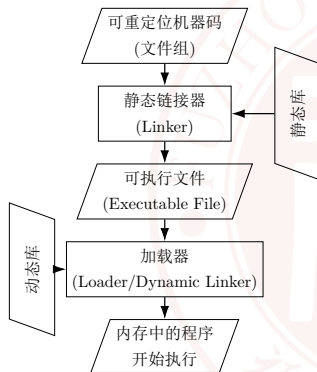
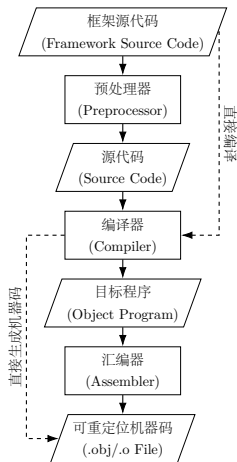
编译系统的构成



单个文件的编译流程

- 预处理：处理文件导入、宏定义等
- 编译：将源代码转换为目标代码，通常为目标的汇编指令
- 汇编：将汇编指令转换为机器代码，输出为可重定位的目标文件（Relocatable Object File）
- 对于某些语言或者部分文件，可以跳过预处理和汇编阶段
- 问题：一个程序有 100000 行代码，如何高效地编译？

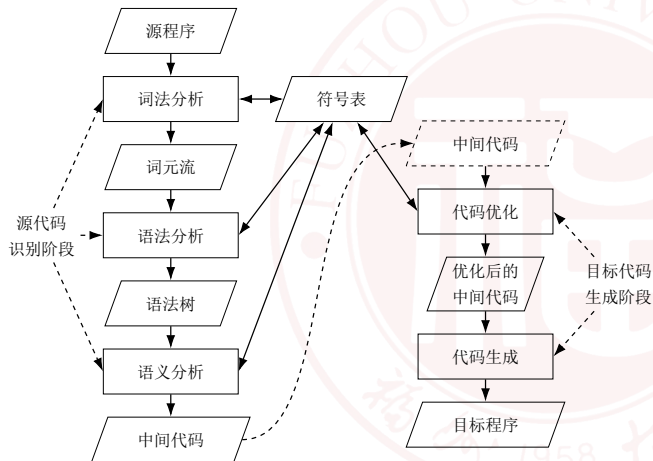
编译系统的构成



现代编译后续流程

- 静态链接程序（Static Linker）：将多个目标文件和库文件链接为单个可执行文件
- 动态链接程序（Dynamic Linker）：在运行时将共享库加载到内存中并处理好跳转

现代编译器的主要阶段



主流开源编译器

GCC GNU 编译器套装（英语：GNU Compiler Collection，缩写为 GCC）是 GNU 计划制作的一种优化编译器，支持各种编程语言、操作系统、计算机系统结构。该编译器是以 GPL 及 LGPL 许可证所发行的自由软件，也是 GNU 计划的关键部分，还是 GNU 工具链的主要组成部分之一。GCC（特别是其中的 C 语言编译器）也常被认为是跨平台编译器的事实标准。

Clang/LLVM Clang 是 LLVM 项目中的一个前端，它是一个 C/C++/Objective-C 编译器，支持多种编程语言。LLVM 是一个模块化、可重用的编译器和工具链技术的集合，提供了优化和生成目标代码的能力。

目录

- ① 课程要求与成绩构成
- ② What - 编译程序是什么？
- ③ Why - 我们为什么需要编译程序？
- ④ Why to learn - 为什么要学习编译原理？
- ⑤ 编译系统概述
- ⑥ 总结与作业



总结与作业

- 本章内容总结



总结与作业

- 本章内容总结
 - 编译程序是将高级语言转换为机器语言的工具



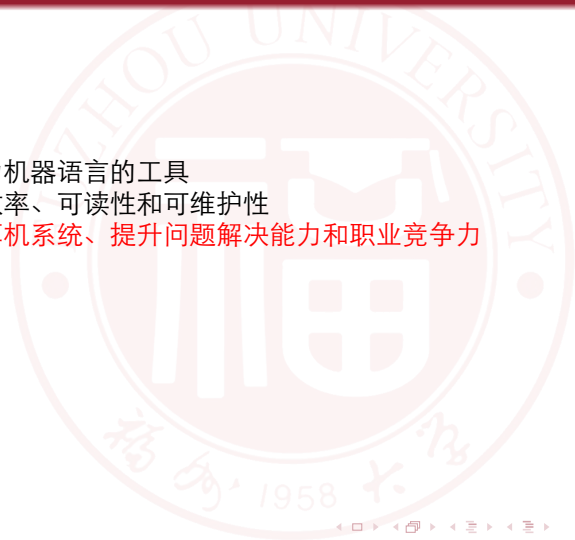
总结与作业

- 本章内容总结
 - 编译程序是将高级语言转换为机器语言的工具
 - 编译程序提高了程序的开发效率、可读性和可维护性



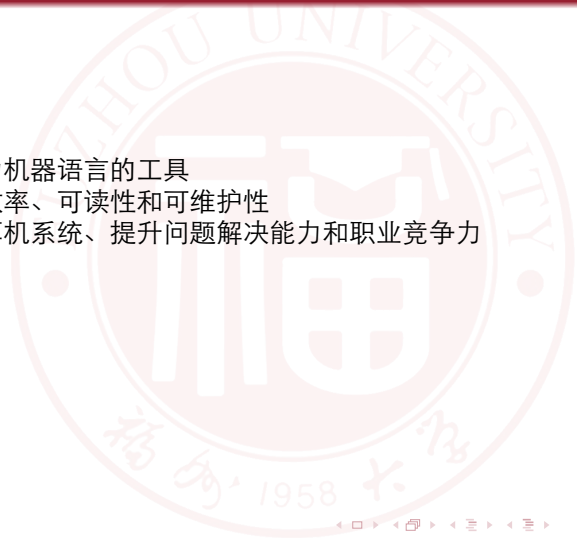
总结与作业

- 本章内容总结
 - 编译程序是将高级语言转换为机器语言的工具
 - 编译程序提高了程序的开发效率、可读性和可维护性
 - 学习编译原理有助于理解计算机系统、提升问题解决能力和职业竞争力



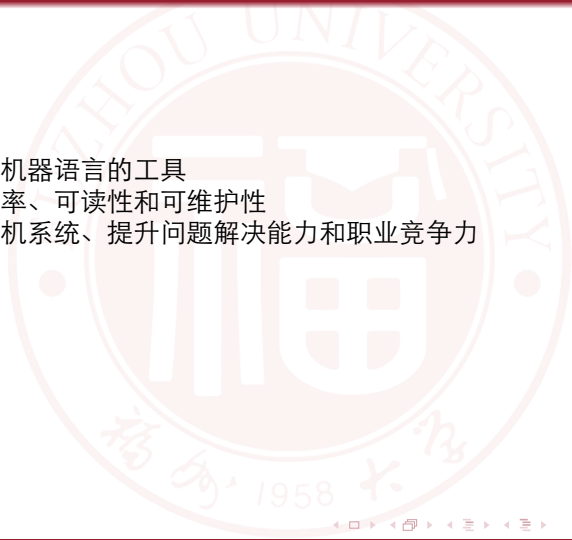
总结与作业

- 本章内容总结
 - 编译程序是将高级语言转换为机器语言的工具
 - 编译程序提高了程序的开发效率、可读性和可维护性
 - 学习编译原理有助于理解计算机系统、提升问题解决能力和职业竞争力
- 课后作业



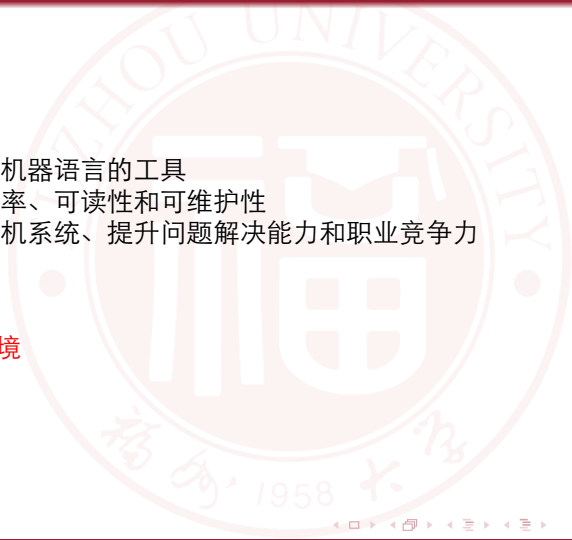
总结与作业

- 本章内容总结
 - 编译程序是将高级语言转换为机器语言的工具
 - 编译程序提高了程序的开发效率、可读性和可维护性
 - 学习编译原理有助于理解计算机系统、提升问题解决能力和职业竞争力
- 课后作业
 - 安装 Debian 虚拟机



总结与作业

- 本章内容总结
 - 编译程序是将高级语言转换为机器语言的工具
 - 编译程序提高了程序的开发效率、可读性和可维护性
 - 学习编译原理有助于理解计算机系统、提升问题解决能力和职业竞争力
- 课后作业
 - 安装 Debian 虚拟机
 - 学习 Java 语言并部署 JDK 环境



总结与作业

- 本章内容总结
 - 编译程序是将高级语言转换为机器语言的工具
 - 编译程序提高了程序的开发效率、可读性和可维护性
 - 学习编译原理有助于理解计算机系统、提升问题解决能力和职业竞争力
- 课后作业
 - 安装 Debian 虚拟机
 - 学习 Java 语言并部署 JDK 环境
 - 用 Java 语言并编写一个支持四则混合运算+sin+cos 函数的计算器程序